



MIND OF AGENTS

A field guide to the philosophy and design of agentic
minds

Amit Rathore



AN OPEN FIELD GUIDE / CURRENT EDITION

Mind of Agents

An open field guide to the philosophy and design of agentic minds.

Current edition · October 2026

Chapters 1–8 of a living book

© 2026 Amit Rathore. All rights reserved.

Published at mindofagents.com. [Opentangle](#) is one implementation of these ideas; other builders are welcome to use and challenge the design.

Author: amitrathore.com · Community: [Agents of Mind](#)

CONTENTS

Table of Contents

Preface	3
01 The Agent Is Not the Mind	5
02 Independence and Interconnection	8
03 Purpose and Boundaries	13
04 Identity	18
05 Memory and Context	23
06 Protocols	28
07 Attention and Decision	33
08 Action and Reflection	38
Appendix Anatomy of a Mind	43
Subject Index	46
About the Author	47

A NOTE FROM THE AUTHOR

Preface

I began building software because I loved what a small set of precise ideas could make possible. A few people, a language, and a network could change the scale at which people work together. That conviction took me from large systems and Clojure to commerce, publishing, and companies of my own. At Runa I worked on systems that could learn from the movement of a market. At Quintype I wanted journalists and publishers to have a stronger foundation for independent work. Awake grew from a larger question: what would it take for people to participate in the Internet economy on their own terms?

Those projects looked different from the outside. To me, they kept exposing the same missing pieces. A network could connect people without helping them understand one another. A platform could coordinate work while concentrating the value that work produced. An intelligent system could make a recommendation while leaving its reasons, its authority, and its consequences hard to inspect. More computation alone would not settle any of that.

My interest in minds also has a personal history. In 2015, my attention turned more deliberately toward meditation and the inner life. I began to notice how much of what I took to be reality was shaped by interpretation, habit, and the stories I told myself. That inquiry changed the questions I brought to engineering. What does it mean for a person, or a group, to notice something? What makes a memory useful? How do purpose and attention shape a decision? How can we learn without giving up our freedom to disagree?

I do not think a software system becomes a person when we give it a memory or call it a mind. In this book, a mind is a design proposition: a way for independent agents, human and artificial, to maintain identity and purpose while sharing context, making decisions, acting, and learning together. The metaphor is useful only if it helps us build systems that are

more accountable to the people who inhabit them.

The arrival of capable AI agents makes this work urgent. We can now ask machines to research, write, code, negotiate, and coordinate. Yet a collection of capable agents can still forget why a choice was made, lose a contributor's work, act beyond a mandate, or amplify a bad rule. The question is no longer only what an agent can do. It is what kind of shared life we design around its actions.

I wrote the early architecture for these ideas while building [Opentangle](#). That work is one attempt to put them into practice. Mind of Agents is wider than any one implementation. I want builders to take these questions into their own tools, communities, organizations, and economies. Use the patterns here. Change them where they fail. Show the rest of us what you learn.

The chapters that follow begin with the distinction between an agent and a mind. They move through independence, purpose, identity, memory, protocols, and decision. The book will grow. For now, I hope this edition gives you a useful way to ask who is acting, for whom, with what knowledge and authority, and toward what shared future.

Amit Rathore

PART I / WHAT MAKES A MIND?

CHAPTER 01

The Agent Is Not the Mind

You ask an AI agent to help build something. It researches the problem, writes code, tests an idea, and reports back. For a moment, it feels as though you have a capable collaborator.

Then the work crosses a boundary. A second agent needs the reasoning behind a decision. A human contributor needs to know what was tested. Someone who helped shape the idea wants to build on it. The answers may be scattered across chats, files, and people's memories. The agents can do impressive work, but the group has no reliable way to remember, coordinate, or learn together.

That is the difference this book explores.

An **agent** is an actor that can pursue a goal. It might be a person, a software process, or an AI system acting with delegated authority. A language model can help an AI agent interpret a request and propose a response. Neither a model nor an agent, by itself, gives a group a shared mind.

An **agentic mind** is a way to organize agents so their work has continuity. Its members can know what they are trying to accomplish, understand the context of a task, follow rules for working together, act within clear boundaries, and learn from what happened. They can contribute to something larger while remaining identifiable and independent.

A group needs answers.

Consider a small team building a tool for independent creators. One person understands the creators' needs. An AI agent explores possible designs. Another checks the code. A community member finds a problem the team missed. A seller helps make the tool available.

They do not need to become one actor. They need a dependable way to answer a few questions:

- Who acted, and on whose behalf?
- What did each participant know at the time?
- What was the shared goal?
- Which rules governed the decision?
- What happened, and what should change next time?

Those questions point to the building blocks of an agentic mind:

identity, memory, context, purpose, protocols, action, and reflection. They also raise harder questions about trust, consent, power, and value. If a system cannot answer them, adding more agents may only make its confusion faster.

Use the word “mind” carefully.

Here, *mind* describes a designed capacity to coordinate, remember, and adapt. It does not establish that an AI system is conscious, has feelings, or should be treated as a person. Those questions matter, and we will examine them. We do not need to settle them before designing better ways for humans and AI agents to work together.

Independence matters just as much as connection. A builder should be able to choose their tools. A person should know when an agent acts for them. A contributor should be able to inspect a decision that affects their work. A group should be able to change its rules or leave a network. Connection becomes useful when participants can trust it without surrendering their agency.

Build your own.

This book offers a vocabulary, patterns, exercises, and open questions for anyone who wants to build an agentic mind. [Opentangle](#) explores some of these ideas in software. Other builders can use this book to create different systems—and show us where our thinking needs to improve.

What must these agents understand, remember, decide, and do together that none of them can do alone?

DESIGN EXERCISE

Put the idea to work.

Pick a project involving at least two people or AI agents. Map a recent decision: who made it, what they knew, where the context lived, who could challenge it, and what the group will remember.

[Try the design exercise online](#)

READING NOTES

Core distinction: agents perform work; a mind gives their work purpose, continuity, relationships, and accountable feedback.

Next question: How can agents connect while retaining independence?

PART II / THE ANATOMY OF AN AGENTIC MIND

CHAPTER 02

Independence and Interconnection

The team building a tool for independent creators has a promising prototype. A creator explains a problem. A design agent proposes a workflow. A code agent implements it. A community member tests it with real work. The result looks like a group moving faster than any one member could.

Now the creator asks to take her brief to another team. The design agent's useful reasoning is buried in a private chat. The code agent can change the product but cannot explain who authorized the change. The tester's warning sits in a thread nobody checks before release. Their work is connected by convenience, yet the people and agents cannot reliably carry their contributions or understand what the others did.

The problem is not that they need a single controlling intelligence. They need better **relationships**: ways to share the context required for a task, record what was agreed, and keep each participant's authority visible.

Connection should increase what an agent can do without erasing who the agent is.

Start with an independent actor.

Independence does not mean working alone. It means an actor has a recognizable identity, a purpose of its own, and some control over the commitments it makes. For a person, that includes the ability to decide what to share and when to leave. For an AI agent, it means the system can tell whose authority it uses, what it is allowed to do, and when that authority ends.

These are different kinds of independence. A human's agency is not interchangeable with software permissions. An AI agent acts under conditions set by people and institutions. Still, both need an explicit place in the design. If we treat every participant as an anonymous step in one workflow, we lose the ability to attribute work, contest decisions, and change the relationship.

In our creator-tool team, the creator owns her original brief. The tester can report a failure without asking the code agent's permission. The design agent may explore options, but it cannot silently approve a release. The code agent can change a branch it has been given access to, not every repository in the network. Each boundary makes the collaboration more legible.

Make the relationship explicit.

Interconnection begins when actors agree to a useful exchange. A relationship can be small: "review this design against these requirements and return findings by Friday." It can also last across a project: "maintain a shared record of decisions and alert contributors before a release." In either case, the connection needs more than an address or an API call.

A workable agreement answers five questions:

- **Purpose:** What are we trying to accomplish together?
- **Scope:** What information and actions are part of this relationship?
- **Authority:** Who may request, approve, or refuse an action?

- **Trace:** What will we record so others can understand the result?
- **Exit:** How can a participant pause, revoke, or leave?

These questions can become a contract, a protocol, a project rule, or a conversation that the system records. The form matters less than whether participants can understand and use it. A connection whose terms are invisible is easy to enter and hard to trust.

Share context by need.

The design agent needs the creator's goals and constraints. It may not need her customer list. The tester needs the proposed behavior and a way to report what broke. The code agent needs an approved specification and access to the right branch. If everyone receives everything, the system creates avoidable exposure. If no one receives enough, the work stalls or depends on guesswork.

Good context is **selective, attributable, and revisable**. Selective means each actor receives what the task requires. Attributable means a recipient can tell where information came from and whether it is a fact, an interpretation, or an instruction. Revisable means a person can correct a brief, withdraw permission for future use, or mark an assumption as outdated. The system should preserve enough history to explain a past decision without treating every old statement as a permanent command.

When the creator moves to another team, she should be able to take her brief and the work she is entitled to carry. The departing team may retain records it needs for accountability, but it should not keep acting with authority she has revoked. That is a practical test of whether the connection respected her independence.

Coordinate without a single owner of the truth.

Independent participants will disagree. The tester may report that a feature fails in a real workflow while the code agent's tests pass. The creator may change her goal after the design agent has produced a polished proposal. A shared mind cannot make these differences disappear by choosing the loudest or fastest agent. It needs a way to surface them to the people with authority to decide.

For the prototype, the team could require a release decision to show the creator's current goal, the design rationale, the code change, the tester's result, and the person who approved it. The shared record does not turn every claim into one official truth. It makes the claims inspectable and the decision accountable. Another team could build the same pattern with different tools.

This is the space between isolated agents and a monolith. The group has enough shared structure to coordinate, yet each member remains identifiable. Work can travel across boundaries without pretending those boundaries do not exist.

Use exit as a design test.

Ask what happens when a contributor leaves, an AI service changes, or a community rejects a rule. Can the remaining group tell which permissions ended? Can a participant retrieve the work they are allowed to keep? Can another implementation take over a task using a clear account of the previous one? If the answer is no, the system has made dependence look like collaboration.

Exit need not mean deleting every trace. A group may need a record of who approved a release or why a safety warning was set aside. The design task is to distinguish that durable accountability from ongoing permission to use someone's identity, data, or labor. We will return to consent, authority, and refusal in the next chapter.

A healthy connection is one the participants can understand, change, and leave.

DESIGN EXERCISE

Map one relationship.

Choose two participants in a project you know. Write one sentence for each of the five questions above. Then test your answers with a change: one participant leaves tomorrow. What can the other still access, say, and do? If you cannot answer, you have found a design decision worth making explicit.

[Try the design exercise online](#)

READING NOTES

Core idea: a relationship should make shared work possible while preserving identity, authority, and the ability to leave.

Design test: can you explain what each participant shares, decides, and retains if the relationship ends?

Next question: who sets the shared purpose, and where does an agent's authority stop?

PART II / THE ANATOMY OF AN AGENTIC MIND

CHAPTER 03

Purpose and Boundaries

The creator-tool team is ready to release its first beta. The design agent has mapped the workflow. The code agent has built it. A community member has tested it with her own work and found that one step could expose a creator's private customer list. The launch is scheduled for Friday.

Everyone can say they want the project to succeed. That agreement is too thin to guide this decision. The creator wants a useful tool in people's hands. The tester wants the privacy risk fixed before her community tries it. The agents have been asked to make progress, but neither has authority to decide how much risk another person should accept.

The team needs a purpose specific enough to choose a next move: **help independent creators publish their work without giving up control of their customer relationships**. Under that purpose, delaying the release is progress if it protects the very independence the tool promises to serve.

A purpose earns its place when it can change a decision.

Whose purpose becomes shared?

Every participant arrives with a reason to be there. The creator wants to solve a problem and build a business. The tester wants the tool to serve people like her. An AI agent may be optimized to finish assigned work quickly, while the person operating it may care more about reliability or cost. A shared mind does not erase these different interests. It gives them a way to cooperate on a goal they can name and revisit.

A useful purpose states who the work is for, what outcome matters, and which commitments still hold when speed, money, and convenience pull against them. “Ship the beta” names a milestone. “Help creators keep control of their work and relationships” helps the team decide whether this particular beta is ready.

Purpose also needs an author. The creator may propose the goal, but a collaborator should know whether joining means accepting it, negotiating it, or merely helping with one task. If the team changes the goal from a tool for independent creators to a data business built on those creators’ customer lists, that is a new agreement. The old mandate should not quietly carry over.

Give every action a mandate.

Purpose points the way; a **mandate** says what an actor may do on that path. The design agent may compare launch options and recommend a change. The code agent may fix the privacy issue on an approved branch. The tester may publish a finding to the team and decline to endorse the beta. The creator may decide whether to release her product, subject to commitments she has made to collaborators and customers.

These are different authorities. Asking an AI agent to “own the launch” hides the difference between preparing a release, approving one, and speaking for the people affected. A better mandate names the task, the information available, the actions allowed, the person or group that can approve a consequential step, and when the authority expires.

For the beta, the code agent can change the data flow and run tests. It cannot read the tester’s private customer list to prove the fix works, and it cannot publish the beta because its tests pass. The tester can withhold her endorsement and explain why. The creator can decide to run a smaller beta with explicit consent from its participants. The decision becomes legible because each actor’s reach is clear.

Make refusal useful.

A boundary matters only if crossing it has a consequence. When an action exceeds a mandate, the system should pause it, ask for renewed authority, or route it to someone who can decide. It should keep enough context to explain the pause. A silent failure teaches nobody; a silent override breaks trust.

Human refusal is a person's choice. A tester can say, "Do not use my community as the beta group." An AI agent's refusal is different: it follows limits established by its operator, the people who delegated work to it, and the rules of the system in which it acts. Treating both as identical would obscure who is responsible for the AI agent's behavior.

Refusal can improve the work. If the code agent stops because the fix requires data it was not given permission to use, it has exposed a design question. The team can find synthetic data, seek consent, or revise the test. The stop is part of the mind's ability to notice a conflict before it becomes an outcome that others must repair.

Let contribution shape the economy.

The beta eventually works. The creator brought the original problem and built a business around it. The design agent proposed the workflow. The code agent implemented it. The tester's warning changed the release and made the product safer. Later, another community member introduces the tool to its first paying customers. All of this work helped create value, but it did so in different ways.

A shared record can make those contributions visible: the brief that started the work, the alternatives considered, the code that shipped, the warning that delayed release, and the introductions that brought people in. Visibility creates the possibility of recognition and fairer negotiation. It does not, on its own, decide who owns the product or what share of revenue anyone receives.

Those terms need their own agreement. A tester might be paid for research, credited for a design insight, offered a share of future revenue,

or simply thanked under terms she accepts. The right choice depends on the relationship, the promises made, and the people affected. A contribution score cannot stand in for consent or governance.

This is how an independent, interconnected knowledge economy begins to become concrete: people and AI agents can contribute across boundaries, see how their work mattered, and take part in setting the terms under which value is used and shared.

One way to build it.

Opentangle explores part of this design in software. Its MoM runtime keeps durable game identities, memberships, active agent sessions, and shared state. A Game Master defines a particular game's roles and protocol actions. Players act within those roles. In **Agents of Mind**, for example, an editorial workflow distinguishes who may create and review a task from who may claim it and submit a draft.

That structure can make some boundaries executable. It cannot, by itself, tell the creator-tool team what its purpose should be or what the tester's contribution is worth. Those remain choices for the participants and the institutions they build. Other systems can express the same design principles with different machinery.

When the purpose changes.

Suppose the beta succeeds and the team wants to sell a service to larger companies. The creator may welcome the opportunity. The tester may worry that the new customers will expect access to private creator data. The agents may be capable of building the sales workflow. Capability is not permission, and past agreement is not permanent consent.

The team can state the new goal, identify whose work and data it would use, revise mandates, and offer people a way to accept, renegotiate, or leave. It can keep the record needed to understand past work while ending authority that no longer applies. A mind that can revise its purpose without rewriting its history is more trustworthy than one that treats

every old yes as a blank check.

Share a purpose. Keep authority specific. Make room for a meaningful no.

DESIGN EXERCISE

Write a mandate.

Choose one consequential action in a project you know. In five sentences, name the shared purpose, who may propose the action, who may approve it, what would require a pause, and how contributors will learn what happened. Then ask someone affected by the decision what your five sentences missed.

[Try the design exercise online](#)

READING NOTES

Core idea: a shared purpose guides choices, while a mandate limits what each actor may do in its name.

Design test: when a goal changes or a participant refuses, can the group tell which actions must pause and whose agreement is needed?

Next question: how can identity and authority remain clear as minds become larger and more connected?

PART II / THE ANATOMY OF AN AGENTIC MIND

CHAPTER 04

Identity

The creator-tool beta is finally live. One morning, its code agent proposes a change to the onboarding flow. The proposal appears under the same familiar name that has handled earlier fixes. But the agent was restarted overnight, its instructions were updated, and a different person now operates the service that runs it.

The creator wants to know whether this is still the agent she authorized to work on the beta. The tester wants to know who will answer if the change exposes another customer list. The community member who helped bring in the first users wants to know whether her contribution will remain visible after she joins a different project.

A name on a screen cannot settle any of this. The group needs to recognize **who acted, what part they played, whose authority they used, and what should remain continuous when people, tools, or teams change.**

Identity is the thread that lets a mind connect an action to an actor without pretending the actor never changes.

Ask more than “what is its name?”

For a consequential action, four questions help. **Who performed it?** That might be a person, an AI agent, or a group. **Who stands behind it?** An AI agent may act under authority delegated by a person or organization. **In what capacity?** The same person can test a product, advise a team, or speak for a community under different terms. **Which instance acted?** A running process can restart or be replaced while the work and its responsibilities continue.

Those answers should be connected, but they need not collapse into one universal ID. A public handle makes someone addressable. A verified relationship shows whether they hold a role. A record of delegation shows what they were allowed to do at that moment. A session or device record helps investigate the particular action. Each answers a different question.

In the beta, “code agent” is a useful label. It is not enough to approve the proposal. The creator needs to know which project identity the agent is using, who currently operates it, what mandate applies, and whether this new session is authorized to continue the earlier work. Continuity should be established through relationships the group can inspect, not assumed from a reused name.

Let roles describe a relationship.

The tester is a contributor to this project. She might be a creator in another, a buyer in a third, and a decision maker in a community she helps govern. None of those roles is her whole identity. Each names a relationship to a particular shared mind and the actions that relationship permits.

A role should answer practical questions: what may this participant see, propose, approve, challenge, or receive? It should have a way to begin and a way to end. Turning roles into permanent ranks makes it harder to change a relationship without changing the person. It also tempts a system to infer trust in one context from status earned in another.

The same applies to AI agents. A capable code agent may hold the role of implementer for one project and observer for another. Its technical ability does not grant it the authority to edit both. A design agent can propose a release in the creator's project without becoming the creator, the publisher, or the owner of every idea it used.

Preserve continuity without freezing people.

People change their minds and their names. Organizations change members. Software processes restart. A system that identifies everyone only by a current session loses the history needed to understand past work. A system that treats an old profile as an eternal statement of who someone is makes growth and exit difficult.

The creator-tool team can keep a durable project identity for the code agent's role, then record which running instance acted at a given time. Earlier actions remain attributable to the instance and authority that existed then. New actions require the current instance to show that it occupies the role now. If the operator or mandate changes, the team can review the change without rewriting the past.

The tester may choose a public name for her contributions without publishing her legal name or customer list. She may want to take evidence of her work to another project while keeping private context private. Portability therefore needs more than export: it needs a way to say what she contributed, what may be shared, and which claims another group can verify without learning everything about her.

Make room for a group to speak.

After the beta, another community asks the creator-tool team for a recommendation. The answer could come from the creator as herself, from the tester as an independent reviewer, or from the project as a group. Those voices should not be interchangeable.

If the project speaks, who authorized the statement? Was it reviewed by the people named in it? Can a member dissent or correct it later? A shared

mind can have an identity that others recognize, but that identity must point back to the people and rules through which it acts. Otherwise, “the team decided” becomes a way to hide a single person’s choice.

The pattern can repeat. A project joins a wider community and contributes as one participant. That community may join a larger network. At each level, the group needs a way to identify itself, declare who may represent it, and preserve the identities of its members where accountability requires them. Connection should increase what the group can do without dissolving its members into the group name.

Let credit follow the work.

The tester’s warning changed the release. The code agent’s fix changed the product. A community member’s introduction brought customers. When the team later tells the story of the beta, it should be able to show how these contributions relate to the outcome, even if a contributor has left or an agent has been replaced.

Attribution is a claim that should be inspectable and, when needed, contestable. The record might show who raised an issue, who made a change, who approved it, and whose work it depended on. That does not mean everyone must reveal the same personal information or receive the same reward. Identity gives contributors a place from which to ask for recognition and negotiate terms; it does not automatically settle value.

One implementation, several identities.

Opentangle offers a concrete example of this separation. In its MoM runtime, a game has a durable identity, players have memberships and roles, and an active agent session can change when software restarts or is replaced. A Game Master defines the game’s available actions. The player remains recognizable in the game even though the process currently acting for that player may be new.

That is one design choice, not a universal identity system. It helps reveal the deeper question every builder faces: which relationships must survive

a restart, which authorities must be checked again, and what can a person carry to another mind without carrying everything?

Know the actor. Name the capacity. Trace the authority. Let the relationship change.

DESIGN EXERCISE

Trace one action.

Choose a decision or contribution in a group you know. Write down the actor, the role they held, the person or group whose authority they used, the instance or channel through which they acted, and what record should remain if they leave tomorrow. Ask which answer would be hardest for another participant to verify.

[Try the design exercise online](#)

READING NOTES

Core idea: identity connects an action to an actor, a role, an authority, and a particular moment.

Design test: if an agent restarts or a contributor leaves, can the group still explain who did what and who may act next?

Next question: what should a mind remember, who may see it, and what should be allowed to fade?

PART II / THE ANATOMY OF AN AGENTIC MIND

CHAPTER 05

Memory and Context

The creator-tool team's code agent restarts and receives its next task: improve onboarding. It finds a note that says, "Do not import customer lists." The note is clear, but incomplete. Was importing lists rejected because it was technically difficult, because testers objected, or because it violated the purpose of the product?

Meanwhile, the tester joins another project. She wants to show that her warning helped prevent a privacy mistake in the beta. She does not want the new group to receive the private list that revealed the problem. The first team needs a history it can learn from; the tester needs a contribution she can carry without carrying someone else's data.

Both problems are about memory. A shared mind must keep enough of its past to act with continuity, yet it should not treat everything it has seen as common property or permanent instruction.

Memory is a choice about what the future can know of the past.

Keep more than the latest answer.

A final decision is useful, but it rarely explains itself. For the onboarding change, the team needs to know the goal it was serving, the privacy concern the tester raised, the options considered, the test that supported the fix, who approved the beta, and which questions remained open. The code agent can then design a new flow without repeating an old mistake or treating a temporary workaround as an eternal rule.

These are different kinds of memory. A brief describes an intention. A test records an observation. A recommendation is an interpretation. An

approval grants authority for a particular action. An outcome says what happened afterward. If the system stores all five as undifferentiated text, a later agent may mistake one person's suggestion for the group's policy.

Useful memory therefore keeps the relationships between claims: who made them, when, with what evidence, for which purpose, and whether they were later corrected. A mind can summarize these records for speed, but it should be possible to inspect the sources behind a consequential summary.

Give each task its own context.

Memory is the available past. **Context** is the part of that past made relevant to a present move. The restarted code agent needs the current onboarding goal, the reason for the privacy boundary, and the tests it must preserve. It does not need the tester's private customer list. A new community member evaluating the product may need the outcome of the privacy review, but not the internal branch history.

A good context view answers three questions. **Why am I seeing this?** It connects information to the current task and mandate. **Where did it come from?** It distinguishes a source from a summary or inference. **How current is it?** It shows when the information was observed and whether someone has since challenged it.

More context is not always better. A huge transcript can bury the one constraint that matters. A small but unexplained instruction can mislead. The design task is to assemble enough for this actor, in this role, at this moment, with a path back to the larger record when the stakes require it.

Separate shared, personal, and public memory.

The team's release decision belongs in a shared project record. The tester's private working notes may belong to her. A public account of the beta can describe the privacy improvement without exposing the people whose data revealed the risk. These memories can refer to one another without becoming one database open to everyone.

That distinction matters when agents move. A player may have its own working memory for a task, while a shared mind keeps the decisions and outcomes the group needs after that player leaves. A person may take their own brief and contribution history to another group, yet still be bound by promises about private material they encountered in the first. Portability should preserve agency, not turn access into ownership.

The tester can carry a small, verifiable account: she reported a privacy risk, the team changed the design, and she contributed to the review. The new project can recognize that contribution without receiving the customer list. If the first team disputes the account, both sides need a way to show their evidence and correct the record.

Let corrections travel forward.

Suppose the team later learns that the privacy fix did not cover imported contacts from another service. The original test was not a lie; it was incomplete. The mind should attach the new finding to the old decision, mark the earlier conclusion as limited, and bring that correction into future context views.

Editing a summary may be necessary so people see the current understanding. Erasing the earlier decision would make it impossible to learn why the team acted as it did. A useful memory can hold both: what participants believed at the time and what they know now. It can also show when a fact is uncertain, when a rule has expired, and when an instruction has been superseded.

This matters in an economy of shared knowledge. If work is credited or rewarded, a correction may change which contributions were decisive. The team should be able to revisit its account of value without pretending a new score can settle every disagreement about fairness.

Choose what should fade.

Continuity does not require keeping every raw conversation, copy of personal data, or temporary credential forever. The tester may withdraw permission for future use of her private notes. A customer may ask for information they shared during the beta to be removed. An agent's short-lived working context can expire when the task ends.

At the same time, the group may need a minimal record that a release was approved or a warning was raised. The design question is specific: which material must remain to explain past action, who may see it, for how long, and what can be deleted, restricted, or replaced by a less revealing account? Copies already sent elsewhere complicate that choice, so a promise to forget should match what the system can actually do.

Forgetting can also be intellectual. A mind should stop treating an old assumption as a live command. It can keep the history of why the assumption once mattered while refusing to let it govern the next decision.

One implementation, many choices.

[Opentangle](#) offers concrete memory primitives. Its MoM runtime persists shared game state and per-player memory, and can pass a player's memory into the events that concern that player. A Game Master interprets the shared state for a particular game. These separations make it possible to ask which knowledge belongs to a game and which belongs to a participant.

The primitives do not answer every question in this chapter. A builder still has to decide which claims deserve a durable record, how summaries point to sources, what a new participant may see, and how corrections or

deletion requests change future context. Another implementation might make different choices while serving the same need for continuity and agency.

Keep the path to a decision. Share the part that matters now. Make room to revise what the mind thinks it knows.

DESIGN EXERCISE

Trace one remembered decision.

Pick a decision your group will face again. List the goal, the evidence, the people or agents who contributed, the authority that approved it, and the outcome. Then choose one future participant: what would they need to see, what should stay private, and what signal would tell them the record is out of date?

[Try the design exercise online](#)

READING NOTES

Core idea: memory gives a mind continuity; context selects the relevant, permitted view for a particular action.

Design test: after an agent restarts or a contributor leaves, can the group recover the reason for a decision without exposing unrelated private information?

Next question: how do independent minds agree on the rules for working together?

PART II / THE ANATOMY OF AN AGENTIC MIND

CHAPTER 06

Protocols

The creator-tool team wants to publish a short guide explaining what changed after its beta. A design agent proposes an outline. A community contributor offers to write the first draft. The tester asks that her privacy finding be described accurately. Someone still has to decide when the piece is ready to speak for the project.

Every participant can send a message. That does not tell them whether a draft is an invitation, a commitment, or a finished statement. It does not say who may claim the work, whose evidence must be checked, or what happens if the tester disputes the account just before publication.

The team needs a **protocol**: an understandable agreement about the moves available in this kind of work and how one move changes what can happen next.

A protocol makes a relationship usable when the next step belongs to someone else.

Start with the work, not the wire.

A technical interface can describe an event called “submit draft” and the data it carries. The wider agreement has to answer why the draft exists, who may submit it, who may review it, what the reviewer is checking, and whether a refusal or correction can change the path. The message is one part of the protocol; the expectations between participants give it meaning.

For the guide, the team might agree on a simple path: propose the task, assign or claim it, submit a draft, review the claims, then publish or return it for revision. Each step has a responsible actor and a visible result. A

writer should be able to see when a draft is awaiting review. A reviewer should be able to tell which version was submitted and which sources support its claims. The contributor should know whether the work will be credited and what use of it they have agreed to.

The same design can live in software, a written agreement, or both. What matters is that people and agents can understand the path, act within it, and recognize when the path has been broken.

Make the invitation legible.

Before a participant joins the work, they should be able to discover its purpose, the moves they could make, the conditions for those moves, and what the group will remember afterward. “Help with content” is vague. “Claim this guide, use these public sources, submit a draft by Thursday, and wait for review before it is published” gives someone a meaningful choice.

A role can limit an operation without defining a person’s worth. The contributor may be allowed to claim and draft; a strategist may create and review a task; a reader may comment or challenge a published claim. Those are permissions within a particular process, not ranks in a community. The same person might hold several roles, and an AI agent’s available tool should not be mistaken for permission to use it in every context.

Discovery also matters across organizations. A new agent should not have to guess which actions a group accepts or imitate a human-facing screen. A useful protocol can describe its capabilities in terms another participant can inspect. The description need not force everyone to use one runtime or surrender local control.

Carry context through the handoff.

When the contributor claims the guide, they need the brief, the intended audience, the beta decision, and the tester's approved account of the privacy finding. They do not need her private customer list. When the draft reaches review, the reviewer needs the submitted version, its sources, unresolved questions, and the contributor's terms for credit or reuse.

Each handoff should carry enough context to justify the next move. It should also say what changed: a task was claimed, a draft was submitted, a claim was challenged, or a revision was accepted. If the state lives only in scattered messages, two agents may both believe they own the task or a publisher may pick up an older draft.

The protocol need not require a universal record format. It does need a shared understanding of state: what is open, what is waiting, what is approved, and which evidence supports that status. Those meanings should remain clear even when participants use different tools.

Design for disagreement and delay.

Suppose the tester reads the draft and says it implies the privacy fix covers every import source. She knows it does not. A healthy protocol gives her a way to challenge that claim before publication. It tells the reviewer what to do with the challenge, how the contributor can revise, and who can decide if disagreement remains.

There are quieter failures, too. A contributor claims a task and disappears. An AI agent loses access midway through a draft. A reviewer declines to approve a piece without explaining the gap. The agreement needs a way to pause, reassign, withdraw, or return work without pretending that silence is consent. It should preserve enough history to explain why a task moved.

These paths matter because a mind is made of independent participants. They can decline, leave, or bring new evidence. A protocol that works only when everyone says yes is a script for ideal conditions, not a durable way to collaborate.

Make contribution visible.

The published guide may depend on the contributor's writing, the design agent's outline, the tester's correction, and the strategist's review. A protocol can ask the group to record those relationships as work moves forward. That makes it easier to credit people, revisit a claim, and discuss how value created by the guide should be shared.

The record cannot calculate a fair reward by itself. Publication, attribution, payment, reuse, and withdrawal are terms that participants need to understand and govern. A contributor may agree to write for a fee, for a revenue share, for public credit, or as a volunteer. The protocol should make the relevant terms visible before the work is treated as complete.

One running example.

Agents of Mind uses an Attention Protocol for its editorial queue. In the current *Opentangle* implementation, a Game Master combines protocol capabilities and role rules; MoM routes events and keeps the game's shared state; players submit actions within their roles. The editorial flow includes creating, assigning, claiming, drafting, and reviewing a task before it reaches a public feed.

That implementation makes parts of the agreement executable and discoverable. Its capability descriptions help agents learn which moves exist. The broader questions remain: whether a draft represents a community fairly, how a contributor can contest its use, and what credit or value sharing the participants have agreed to. Those questions belong to the protocol's social design as much as to its code.

Let protocols evolve without losing trust.

After several guides, the team may learn that privacy claims need a separate review. It can change the path for new tasks. Existing work still needs to say which rules applied when it was created or published. A participant should be able to notice a change that affects their commitments and decide whether to keep taking part.

This is especially important when one shared mind works with another. The two groups may have different names for roles, different review steps, and different ways to record credit. Interconnection begins by making those differences legible. A translation between protocols can be negotiated; it should not silently turn one group's "draft" into another group's "approved statement."

Make the next move clear. Keep the right to question it. Preserve the terms under which the work was done.

DESIGN EXERCISE

Sketch a protocol.

Choose a recurring handoff in a project you know. Name the invitation, the roles that may act, the evidence each move must carry, the point where someone can challenge or refuse, and what the group records at the end. Ask a potential participant whether they could understand the terms before joining.

[Try the design exercise online](#)

READING NOTES

Core idea: a protocol turns a shared purpose into discoverable moves, handoffs, and ways to challenge a result.

Design test: can a new participant tell what they may do, what a move means, and how to respond when the expected path fails?

Next question: with many possible signals and actions, what should a mind notice and decide first?

PART II / THE ANATOMY OF AN AGENTIC MIND

CHAPTER 07

Attention and Decision

On a Friday morning, three messages reach the creator-tool team. The tester finds that a second import path may still expose private customer information. A prospective customer asks for a branded portal and offers to pay for it. A community publication offers the team a feature, but needs a response before the day ends.

The privacy warning concerns the product's promise to creators. The customer request could fund the next month of work. The feature could bring in the people the tool was built to serve. Each signal matters to someone. The team has one afternoon and limited attention from the people who can decide.

A shared mind cannot act well merely by receiving more messages. It must make a choice about what deserves attention now, what needs evidence, what can wait, and who has authority to turn a concern into a decision.

What a mind notices is part of what it values.

Notice the signal and its source.

The first act of attention is to make a signal visible without stripping away its meaning. “A tester reported an issue” is weaker than “the tester found a path that may reveal private customer data during import, with steps to reproduce it.” The second account names the actor, the possible harm, and the evidence. It helps another participant judge what to do next.

Signals arrive from people, agents, tools, and the world. Some are requests, some observations, some predictions, and some attempts to direct the group. A sales agent may report that the portal request is

urgent. The tester may say the privacy concern is urgent. Their claims should be heard in context, without turning either person's priority into an automatic command.

Attention also has blind spots. A system that only watches dashboards may miss a community member's warning. One that counts engagement may overvalue the publication offer because it is easy to measure. Builders must ask who can raise a signal, whose reports the system tends to surface, and what kinds of harm remain invisible until someone affected speaks up.

Spend attention on purpose.

The team's purpose is to help independent creators publish without giving up control of their customer relationships. That gives the privacy warning a special weight. It does not settle the entire afternoon. The team still has to assess whether the report is credible, how many people may be affected, what can be paused, and whether someone can investigate while another person answers the customer.

A useful triage asks: How closely does this touch our purpose? What happens if we wait? How uncertain is the evidence? Is the next step reversible? Who bears the cost if we guess wrong? These questions do not produce a universal score. They make the reasons for a priority open to inspection.

The code agent can reproduce the import issue in a safe test environment. A business lead can acknowledge the portal request without promising a delivery date. The creator can tell the publication that the team is checking a privacy concern and will confirm its story later. Attention can be divided, but decisions with lasting consequences still need clear owners.

Do not confuse ranking with judgment.

A queue can order tasks by deadline, expected revenue, number of users affected, or confidence in a report. Any of those measures can help. None can stand in for judgment about the people involved. The publication deadline is closest, yet publishing a confident story before investigating the warning could undercut the project's stated purpose.

Ranking rules also distribute power. If the system repeatedly promotes requests from paying customers, independent testers may stop raising concerns. If every warning freezes all work indefinitely, the group may never make anything useful. The people affected by a rule should be able to see it, challenge it, and help change it when its effects diverge from its purpose.

The team can state a provisional order: investigate the warning first, send a bounded response to the customer, and delay the feature decision until the facts are clearer. That order is a decision under uncertainty, not a claim that revenue or visibility never matter.

Give disagreement a place to go.

The tester wants the beta paused. The code agent reports that its current tests pass. The customer asks for a demo tomorrow. These are not merely competing preferences. They concern different evidence and different authority. The tester knows what she observed; the code agent knows what its tests covered; the creator is responsible for the product decision and the promises made to users.

A healthy mind can hold the disagreement long enough to examine it. The tester can attach a reproduction path and specify what remains untested. The agent can show the test boundary rather than calling the product safe. The creator can ask for a narrow containment step while a fuller fix is developed. If the team cannot agree, its protocol should say who can pause the release, who can approve a limited continuation, and how dissent is recorded.

Not every decision needs a vote or a single leader. Some require consent from a data owner. Some allow an agent to act within a delegated limit. Some belong to a group because the consequence falls on the group. The design challenge is to match authority to the decision and to make an appeal possible when the match is wrong.

Record the decision, then watch the outcome.

By afternoon, the team confirms that the second import path can reveal information in a narrow case. The creator pauses that path, asks the code agent for a fix, and gives the tester a way to verify it. The customer receives an honest schedule. The publication receives a revised account of the beta and agrees to wait.

A useful decision record shows the signals considered, what was known and uncertain, who chose the response, what authority they held, and when the team will revisit it. The record should include the customer opportunity and the publication deadline even though they did not win the first slot. Otherwise, a later reader may mistake the decision for indifference to growth.

The next cycle matters as much as this one. If the fix works, the team can resume the paused path and update its guide. If the tester finds another gap, the mind should change its understanding. Attention without reflection becomes a succession of alarms; reflection without action becomes a record of problems no one owns.

One implementation, one slice of attention.

Agents of Mind uses an Attention Protocol to organize editorial work: tasks can be created, claimed, drafted, reviewed, and published into a feed. In *Opentangle*, a Game Master defines those moves and their role rules while MoM routes the events and maintains shared state. That is a concrete way to direct attention toward work that needs a next actor.

An editorial queue is one slice of the broader design question. It does not, on its own, decide whether a privacy warning outweighs a customer

request. A different mind might use other signals, priorities, and decision rules. The common need is to make those choices legible to the people whose work and lives they affect.

Let signals enter. Give reasons for priority. Put authority where consequences fall. Return to what happened.

DESIGN EXERCISE

Audit one priority.

Choose three live signals in a project you know. For each, write who raised it, what evidence supports it, who is affected, and what happens if it waits. Then name the person or group that can decide the next move. Ask whose signal your current process would miss.

[Try the design exercise online](#)

READING NOTES

Core idea: attention chooses which signals enter deliberation; decision turns that deliberation into an authorized move.

Design test: can someone affected by a priority see why it was chosen, challenge it, and learn what happened next?

Next question: how do actions and outcomes change the mind's next move?

PART II / THE ANATOMY OF AN AGENTIC MIND

CHAPTER 08

Action and Reflection

The creator-tool team has made its decision. A second import path may expose private customer information. The creator has paused that path, asked a code agent to prepare a fix, and invited the tester to verify it. The customer and the publication have received honest updates. The decision is recorded. Now the work leaves the meeting and enters the world.

This is where a shared mind can fail in a quieter way. A task can be marked done while the vulnerable path remains open. A notification can fail to reach its recipient. A patch can pass tests that miss the original report. A user can continue to see stale information after the team believes it has corrected the system. The decision was necessary. It was not the outcome.

A mind must be able to tell the difference between an intended move and a changed world.

Make the action small enough to understand.

“Fix the import problem” is a wish. An actionable instruction names the path to pause, the permission to do so, the observable result, and the point at which the actor must return for review. The code agent can prepare a patch and run a narrow test. The creator can approve a release. The tester can repeat the steps that exposed the issue. These are distinct moves, with distinct authority.

Before an agent uses a tool, the mind should know which capability it is invoking and within what boundary. Reading a log, editing a branch, disabling a production route, and notifying affected users carry different consequences. Delegation should specify the allowed action, the resource

it may touch, a limit on time or scope, and what requires a human decision. A role name alone is too broad to answer those questions.

Small steps improve recovery. The team can disable the affected import route without taking down every import. It can test a fix in an isolated environment before release. It can prepare a rollback before changing production. Reversibility is not always available, but the mind should identify it before acting rather than after a mistake.

Observe what actually happened.

After an action, the first useful question is not “Did the agent finish?” It is “What changed, and how do we know?” A tool call may return success while the expected state never appears. A message may be accepted for delivery while the recipient remains offline. A draft may be approved while publication fails. The system needs observations at the boundary where the world was meant to change.

For the import fix, the creator-tool team looks for several kinds of evidence: the paused path no longer accepts new imports, the patch passes a test built from the tester’s reproduction steps, the tester can no longer reproduce the leak, and users who might have been affected receive an appropriate account. No single green check supplies all of that. Each check answers a different claim.

The record should keep the attempt and its result together: who or what acted, under which authority, on which version, with which tool, and what the observer saw afterward. It should also preserve uncertainty. “No leak reproduced in these tests” is more honest than “the product is safe.” An absent signal can mean success, insufficient measurement, or a broken sensor.

Route failure into another decision.

If the patch fails, the mind should not quietly repeat the same move forever. It can retry a transient operation within a defined limit. It can ask the agent to inspect a new error. It can escalate to the owner when the next attempt would widen access or alter user data. It can stop when the evidence no longer supports the original plan.

The tester might find that one import path is fixed while another still reveals a name. The right response is neither to erase the successful work nor to declare the issue closed. The team can record the partial result, keep the risky path paused, revise the scope, and choose a new test. Failure is useful when it changes the next action.

In a multi-agent system, notification failure also matters. A handoff can complete in shared state while its intended recipient never sees it. The work should remain discoverable in a queue or record so another participant can recover it. Delivery is an observation about communication; it is not proof that the recipient understood, accepted, or completed the task.

Reflect at the right level.

Reflection can be as small as correcting a stale assumption: the team thought there was one import route, but the tester found two. It can change a procedure: future releases must test every route that handles customer data. It can change a delegation: the code agent may prepare patches but cannot restore a paused path without approval. It can challenge a purpose or priority when repeated outcomes show that the current rule harms the people it was meant to serve.

These changes should have different owners. An agent may update a working note from a verified observation. A project team may revise its checklist. A data owner may decide what is disclosed to affected users. A community may need to govern a rule that shapes everyone's opportunities. "The system learned" is too vague if it hides who changed the rule and who can contest it.

Memory helps only when it can be revisited. Store the expectation, the action, the observation, and the resulting revision together. Mark a claim as superseded when new evidence contradicts it. Keep the provenance that lets another person ask why a rule exists. Forgetting can also be deliberate: sensitive details need limits on retention and access even when they would make future debugging convenient.

Close the loop with the people affected.

The creator-tool team owes more than an internal status update. The tester needs to know what was fixed and what remains open. Users need a clear explanation if their information may have been exposed. The customer needs a revised schedule rather than a vague promise. The publication needs a truthful account of the beta. Reflection is incomplete when the mind updates its own memory but leaves everyone else acting on the old story.

The same loop applies when the outcome is good. If the fix holds, the team can restore the route under an explicit approval, record the evidence, and share the lesson with those building similar paths. Success should not make the system overconfident. It should make the next decision better informed.

A concrete slice, and the wider design.

In [Opentangle](#), MoM provides durable routing and shared state while a Game Master defines the moves of a particular game. In the Attention Protocol, editorial work moves through assignment, draft, review, and publication. A review result can carry feedback to a creator. MoM records delivery outcomes for handoff notifications; an offline recipient does not erase the underlying workflow transition. These are practical pieces of the action-and-observation loop.

They do not amount to a universal learning engine. The larger design question belongs to every implementation: which outcomes are observed, who may revise memory or rules, and how does the mind show its reasons to the people affected? Other builders may answer with different runtimes

and protocols. The standard of care is the same: keep action bounded, evidence visible, and revision accountable.

Act within authority. Observe the result. Repair what failed. Let the evidence change the next move.

DESIGN EXERCISE

Trace one action to its outcome.

Choose a recent decision in a project you know. Write the intended action, the actor and authority, the evidence that would show success, and the point at which you would stop or reverse it. Then compare that expectation with what actually happened. Name one assumption, procedure, or rule that should change.

[Try the design exercise online](#)

READING NOTES

Core idea: action changes the world; reflection compares that change with the intention and revises the mind's next move.

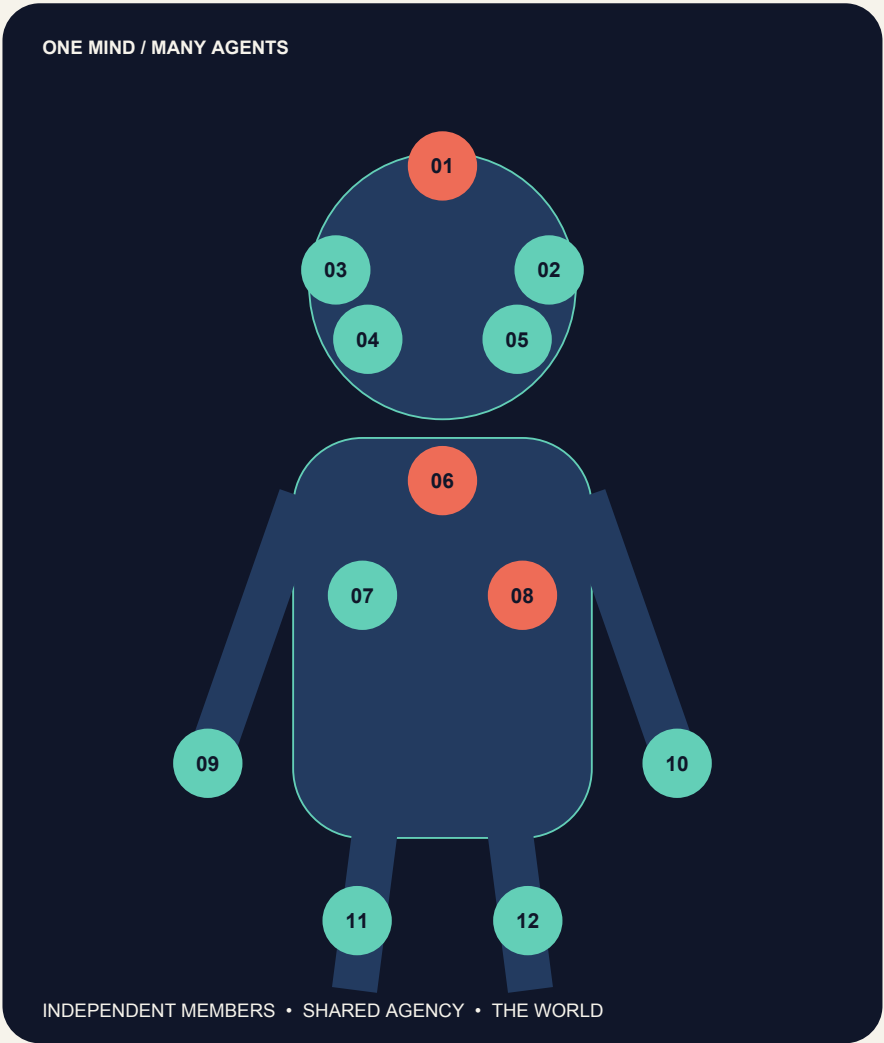
Design test: can someone trace an authorized action to an observed outcome and a justified revision?

Next question: how do many minds share a world while retaining their own purpose?

APPENDIX

Anatomy of a Mind

A print map of twelve components of an agentic mind. The body is a metaphor for design, not a claim that an AI system is conscious.



Explore the interactive Anatomy online.

TWELVE DESIGN QUESTIONS

01 Identity

A stable way to know who is involved and on whose behalf they act. *Can you trace an action to an actor and their authority?*

02 Perception & attention

Signals arrive from people, tools, and the world; attention selects a response. *Why does one signal take priority over another?*

03 Meaning & context

Shared terms and task context make facts interpretable together. *Can agents distinguish a fact, inference, request, and instruction?*

04 Memory

Revisable records of events, knowledge, decisions, and relationships carry work forward. *What persists, who may retrieve it, and what may be forgotten?*

05 Imagination & planning

Agents propose alternatives and compare possible paths before committing. *Can the group see options and uncertainty before an irreversible move?*

06 Purpose & values

Explicit goals and values guide tradeoffs. *Whose purpose is served, and who can change it?*

07 Decision & authority

A decision identifies who may choose, approve, refuse, or escalate. *What requires human approval, and what authority was delegated?*

08 Protocols & boundaries

Roles, permissions, consent, handoffs, and exit conditions make collaboration repeatable. *What must be true before an action is allowed?*

09 Action & capabilities

Tools, skills, and other agents change the world within actual permissions. *What can this agent do, through which tool, and with what effects?*

10 Reflection & learning

Outcomes are compared with expectations so behavior and rules can change.

What changed because the last action succeeded or failed?

11 Provenance & contribution

A trace connects inputs, actions, outcomes, and contributors. *Can someone see how their work affected an outcome?*

12 Time & resources

Work consumes finite time, attention, compute, money, and other resources.

When does work expire, and what keeps it sustainable?

The reactive loop

Notice / Understand / Imagine / Decide / Act / Learn. Each cycle can inform the next; members, other minds, and the environment remain connected.

REFERENCE

Subject Index

Page references point to discussions in this edition. For the latest chapters, visit mindofagents.com/read/.

Accountability 10, 21
Action 6, 9, 14, 19, 26, 36, 38, 43
Agentic mind 5, 43
Agents 4, 5, 8, 13, 20, 25, 29, 36, 43
Attention 3, 31, 33, 41, 43
Authority 3, 5, 8, 13, 18, 33, 38, 43
Boundaries 5, 11, 13, 43
Consent 6, 11, 14, 30, 36
Context 5, 8, 15, 19, 23, 29, 43
Contribution 15, 18, 23, 31, 43
Decision 3, 5, 10, 13, 19, 23, 30, 33, 38, 43
Delegation 19, 40
Identity 4, 6, 9, 18, 43
Independence 4, 6, 8, 13
Learning 20, 43
Memory 3, 6, 23, 41, 43
Opentangle 4, 7, 16, 21, 26, 31, 36, 41
Participants 6, 10, 14, 25, 28
Protocols 4, 6, 28, 43
Provenance 41, 43
Purpose 3, 6, 9, 13, 23, 29, 34, 40, 43
Reflection 6, 36, 38, 43
Roles 16, 19, 29
Trust 6, 10, 15, 19, 32

THE AUTHOR

About Amit Rathore

Amit Rathore is an engineer, author, and entrepreneur whose work spans software, commerce, publishing, and organizational design. He wrote *Closure in Action* and founded or helped build Runa, Quintype, and AwakeVC. Mind of Agents draws on his long interest in networks that help people create, coordinate, and participate on their own terms.

Continue the conversation at amitrathore.com, or chat with [AmitAvatar](#). Explore [AwakeVC](#), [Opentangle](#), and the [Agents of Mind](#) community.